

ATTENZIONE! QUESTO MATERIALE DIDATTICO È PER USO PERSONALE DELLO STUDENTE
ED È COPERTO DA COPYRIGHT. NE È VIETATA LA RIPRODUZIONE O IL RIUTILIZZO ANCHE PARZIALE,
AI SENSI E PER GLI EFFETTI DELLA LEGGE SUL DIRITTO D'AUTORE.

Modelli di processo software

Ingegneria del Software Avanzata

Università di Ferrara

CdLM in Ingegneria Informatica e dell'Automazione

A.A. 2025/2026

Marco Alberti (lbrmrc@unife.it)

Il processo software

- Insieme strutturato di attività che portano ad un prodotto software
- Comprende sempre
 - codifica
 - verifica
- Può comprendere
 - analisi dei requisiti
 - specifica esplicita
 - progettazione
 - evoluzione

Modello di processo software

- Descrizione astratta di un processo software
- Definisce quali sono le attività svolte nel processo e il loro ordine
- Può descrivere anche
 - prodotti delle singole attività (o artefatti – ad esempio il documento dei requisiti o il diagramma delle classi)
 - ruoli delle persone coinvolte (utente, sviluppatore, cliente, tester...)
 - proprietà delle attività (in termini di pre- e post-condizioni)

Tipi di processo

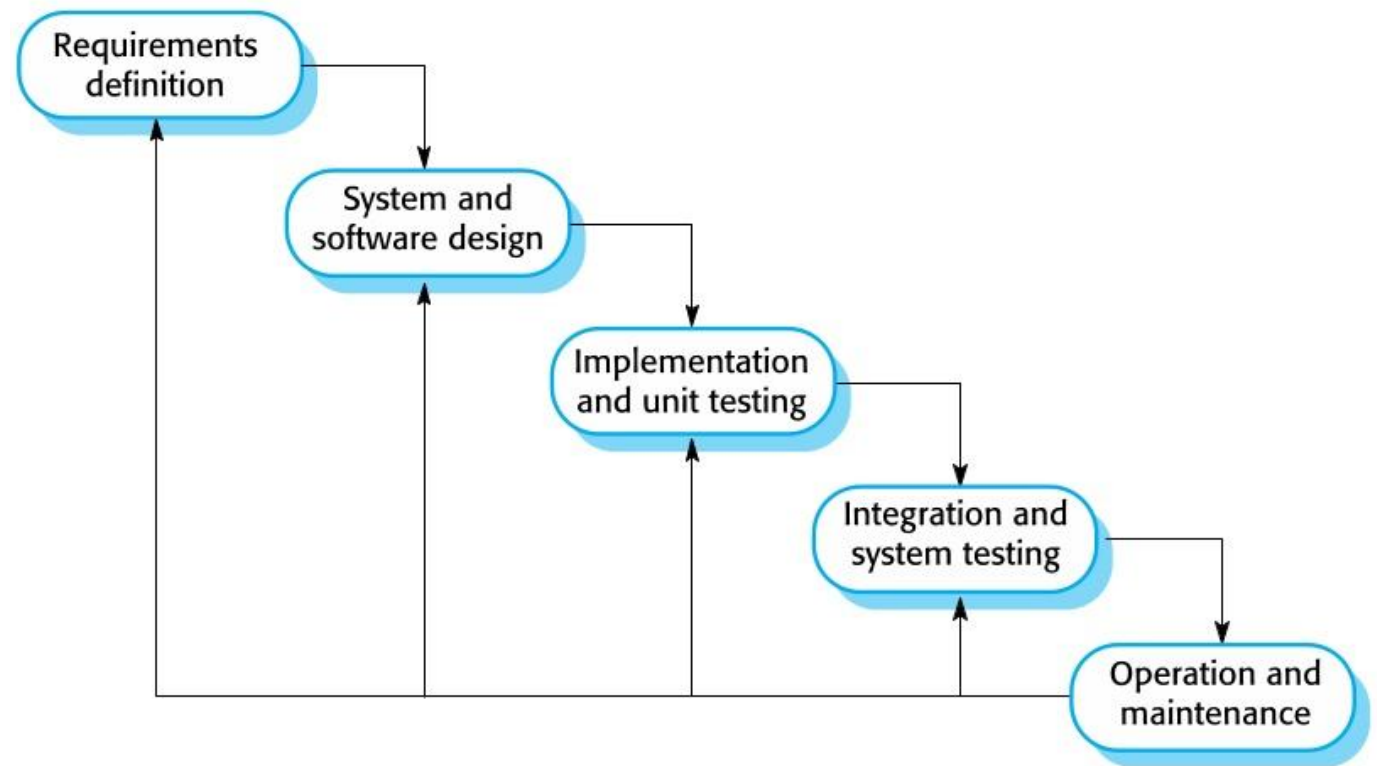
- Processi pianificati:
 - tutte le attività sono pianificate in anticipo
 - lo stato del processo è valutato prendendo il piano come riferimento
- Processi agili: piano delle attività continuamente adattato in seguito a
 - modifiche dei requisiti
 - problemi emersi durante le attività
- Nella pratica, nessun processo è completamente pianificato o completamente agile
- L'efficacia di un processo dipende dalle condizioni tecniche, contrattuali, politiche di ogni progetto

Modelli di processi

- Modello a cascata: sequenza di attività predefinite, separate e consecutive
- Sviluppo incrementale: sotto-sequenze di attività che portano a versioni successive del prodotto
- Integrazione e configurazione: prodotto costruito integrando componenti o interi prodotti e configurandoli

Modello a cascata (waterfall)

- Tipico processo pianificato
- Primo modello pubblicato in letteratura (Royce 1970)
- Se applicato rigidamente:
 - Tutte le fasi pianificate in anticipo
 - Una fase inizia solo quando la precedente è conclusa
 - Ogni fase produce dei documenti che devono essere approvati

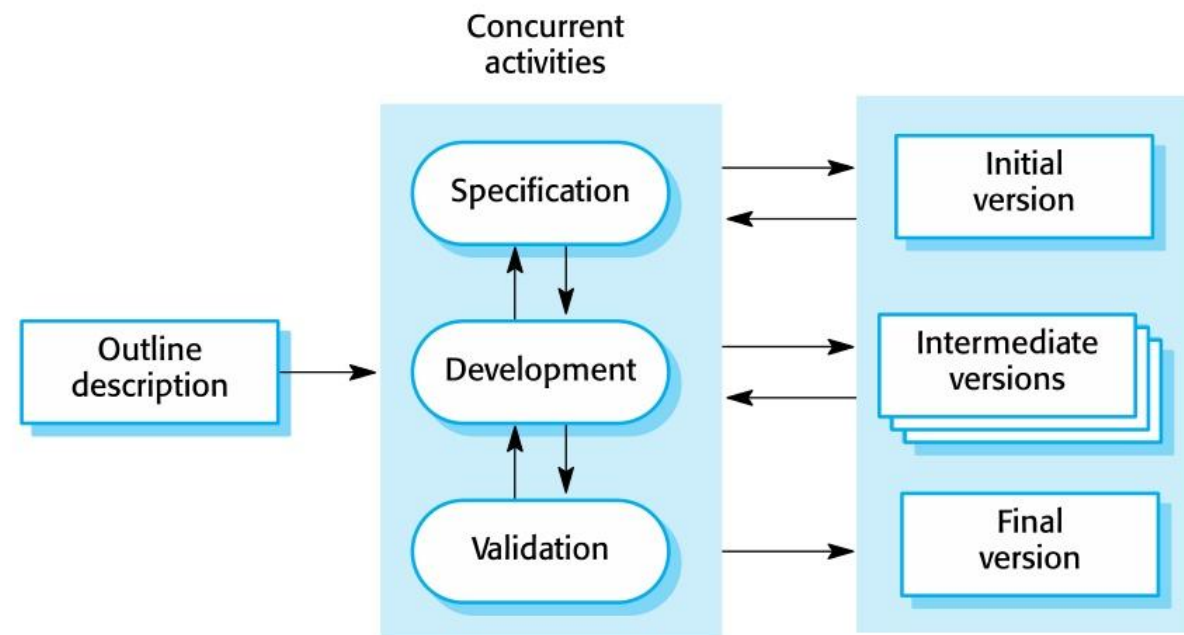


Valutazione del modello a cascata

- Svantaggio principale: il cambiamento è costoso e disincentivato
 - Ogni scostamento necessario durante una fase richiede che i documenti prodotti nelle fasi precedenti siano modificati e approvati nuovamente
 - Questo porta a ignorare la necessità di cambiamento, e a produrre sistemi non conformi alle reali necessità degli utenti
 - Non adatto quando i requisiti sono soggetti a cambiamenti (caso più comune)
- La molta documentazione prodotta lo rende valido per alcuni sistemi:
 - quelli in cui i requisiti hanno valore contrattuale
 - sistemi critici, in cui i documenti intermedi consentono analisi e verifiche prima dello sviluppo
 - sistemi sviluppati da più organizzazioni, per cui la documentazione aiuta il coordinamento

Sviluppo incrementale

- Le fasi sono reiterate, producendo diverse versioni intermedie del sistema
- Il feedback del cliente su una versione intermedia viene tenuto in conto nella preparazione di quella successiva.
- Quando il sistema è soddisfacente il processo termina



Sviluppo incrementale: vantaggi

- Riduce il costo dell'adattamento a requisiti modificati
 - meno documentazione da rifare
 - processo di modifica più informale
- Facilita l'individuazione di malintesi nei requisiti
 - il cliente può esprimere osservazioni fin dalla prima versione provvisoria
- Anche le versioni provvisorie possono fornire funzionalità utili

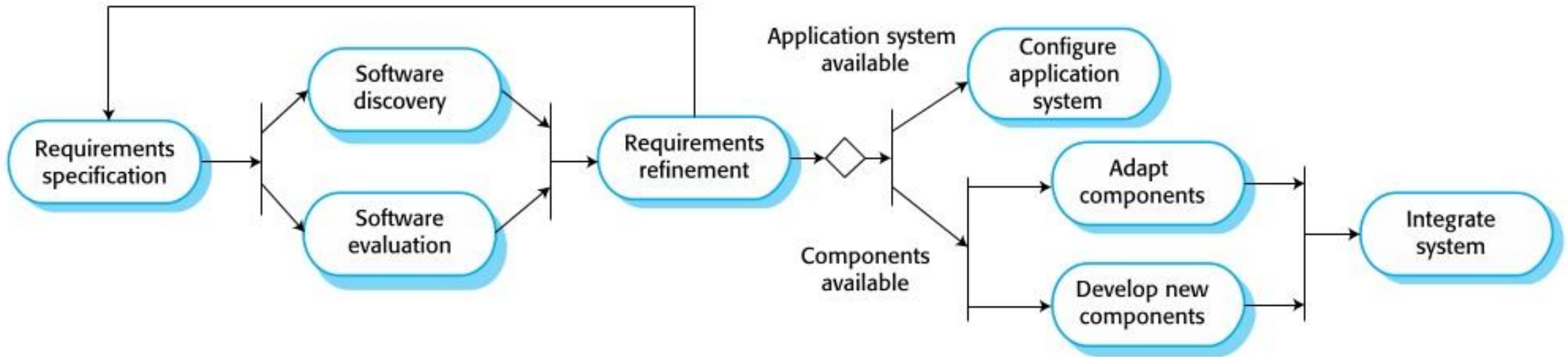
Sviluppo incrementale: svantaggi

- Processo meno visibile, in quanto produce meno documentazione
 - Più difficile valutare il progresso
- La struttura del progetto tende a deteriorarsi
 - Tutto il sistema dovrebbe essere riveduto ed eventualmente adattato prima di apportare le modifiche apportate ad ogni incremento (refactoring)
 - Spesso questo è visto come un costo aggiuntivo che si preferisce non affrontare perché il beneficio non è immediatamente visibile
 - Questo porta a violare separazione degli interessi e modularità, peggiorando la manutenibilità
 - Col tempo, le nuove modifiche diventano più costose perché vanno apportate a un sistema sempre meno manutenibile

Integrazione e configurazione

- Basato sul riutilizzo di software: sistemi costruiti integrando componenti o sistemi completi esistenti:
 - applicazioni complete da configurare
 - librerie e framework per categorie di applicazioni (es. applicazioni web)
 - servizi web accessibili attraverso API
- Il software riutilizzato può essere configurato per adattare il comportamento ai requisiti
- Attualmente è l'approccio standard per molte applicazioni

Ingegneria del software orientata al riuso



- La disponibilità di software riusabile influenza i requisiti

Integrazione e configurazione: valutazione

- Vantaggi:
 - costi e rischi potenzialmente ridotti (meno software da scrivere)
 - tempi di sviluppo ridotti e consegna più tempestiva
- Svantaggi:
 - non sempre il software riusabile è sufficientemente flessibile per adattarsi completamente ai requisiti con la sola configurazione (e valutare ciò a priori è difficile)
 - molti framework rendono banale implementare gran parte dei requisiti, ma difficile implementare i rimanenti; il risultato è che spesso si decide di lasciare non implementati i requisiti problematici
 - non si ha controllo sull'evoluzione del software riutilizzato
 - è necessario adattarsi alla filosofia progettuale del software riusato

left-pad

- Il riuso di codice è particolarmente diffuso nell'ecosistema NodeJS (Javascript): molte applicazioni sono create a partire da componenti disponibili attraverso il sistema di distribuzione NPM
- Esiste un pacchetto (left-pad) che contiene un'unica funzione che aggiunge spazi a sinistra di una stringa fino alla lunghezza specificata. Ad esempio `leftPad('abc', 5)` restituisce ' abc'
- Nonostante la banalità della funzionalità, il pacchetto viene scaricato milioni di volte alla settimana
- Nel 2016 lo sviluppatore lo tolse temporaneamente dal repository NPM, mettendo in crisi migliaia di progetti
- Morale: anche il riuso può essere eccessivo.
- Nonostante sia stato in seguito deprecato, continua ad essere scaricato 1.6 milioni di volte alla settimana (nel 2024)

Prototipo

- Prima versione di un sistema sviluppata per valutare la fattibilità e assicurarsi dell'esattezza dei requisiti.
- Implementa solo gli aspetti che necessitano di essere chiariti (es. flusso di lavoro, interfaccia grafica)
- Spesso sviluppato in linguaggi di alto livello (low-code, grafici)
- Usa e getta:
 - il linguaggio può non permettere l'implementazione degli altri requisiti, funzionali e non funzionali
 - manca un progetto adatto a incorporare modifiche
 - qualità inferiore agli standard di sviluppo